

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I Atchaya Chandran R(192521394) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Atchay

Question 1: Secure Password Storage in Web Applications

1A. Why Storing Plaintext Passwords is Insecure

Storing user passwords in plaintext exposes an application to severe security risks:

- **Immediate System-Wide Exposure:** In the event of a database breach or SQL injection attack, attackers instantly gain full access to every user account without needing to crack encryption keys or hash values.
- **Credential Stuffing Risks:** Most users reuse passwords across multiple services. A single database leak compromises the user's identity across independent external platforms (email, banking, social media).
- **Insider Threats:** Internal staff, database administrators, or developers with access to database backups or logs can read and misappropriate user credentials directly.
- **Non-Compliance:** Plaintext storage violates major regulatory frameworks, such as PCI-DSS, GDPR, and ISO/IEC 27001, resulting in severe legal and financial penalties.

1B. Secure Method & Justification

- **Recommended Method: Salted Password Hashing using Argon2id or bcrypt** with an adaptive work factor.
- **Implementation Steps:**
 1. **Generate a Unique Salt:** For every user account creation or password update, generate a cryptographically secure random value (a "salt") of at least 128 bits.
 2. **Combine and Hash:** Concatenate the plaintext password with the unique salt.
 3. **Apply a Key-Stretching Hash Function:** Pass the salted password through **Argon2id** or **bcrypt** configured with a high computational cost factor (work factor).
 4. **Store Result:** Store the resulting hash along with the salt in the database.
- **Justification:** Cryptographic hash functions are one-way (non-reversible). The addition of a unique **salt** prevents precomputed **Rainbow Table attacks** and ensures two users with identical passwords yield completely different hash outputs. Key-stretching algorithms (Argon2id/bcrypt) introduce deliberate memory and CPU

delays, making offline brute-force and GPU/ASIC cracking attempts computationally infeasible.

Question 2: Digital Signature for Document Authentication

2A. How Digital Signatures Work in Document Authentication

Digital signatures utilize **Asymmetric Key Cryptography** (Public Key Infrastructure - PKI) to authenticate electronic documents:

[Sender Side]

Document ---> [SHA-256 Hash] ---> Hash Value ---> Encrypt with Private Key ---> [Digital Signature]

[Recipient Side]

Received Signature ---> Decrypt with Sender's Public Key ---> Decrypted Hash

Received Document ---> Recalculate SHA-256 Hash ---> Calculated Hash

Compare Decrypted Hash vs. Calculated Hash (Match = Valid Integrity & Authenticity)

1. **Hash Generation:** The sender calculates a fixed-length cryptographic hash of the document using a strong hashing algorithm (e.g., SHA-256).
2. **Signature Creation:** The sender encrypts this hash using their **Private Key**. The output is the digital signature attached to the document.
3. **Verification:** The recipient uses the sender's **Public Key** (validated via a trusted Certificate Authority) to decrypt the digital signature and reveal the original hash.
4. **Comparison:** The recipient calculates a new hash of the received document. If the calculated hash matches the decrypted hash, the signature is valid.

2B. Justification for Integrity and Non-Repudiation

- **Data Integrity:** Cryptographic hash functions exhibit the **avalanche effect**—even a single modified character or space in the document

completely alters the calculated hash. A hash mismatch alerts the recipient instantly that the document was altered in transit.

- **Non-Repudiation:** Asymmetric cryptography relies on key pair exclusivity: only the sender holds their private key. Because only the matching private key could have generated a signature that successfully decrypts using the sender's public key, the sender cannot deny having created and sent the document.

Question 3: Preventing Replay Attacks in Online Transactions

3A. How Replay Attacks Occur

A replay attack occurs when an unauthorized entity eavesdrops on network traffic, intercepts a valid transmission (e.g., a financial request payload), and maliciously re-sends the exact same transaction payload to the server later.

- **Mechanism:** The attacker does not need to decrypt or alter the payload. Because the original transaction was legitimate and validly signed, the target server accepts the re-transmitted data as a new, legitimate request—causing duplicate payments or unauthorized actions.

3B. Cryptographic Technique & Justification

- **Recommended Technique: Cryptographic Nonces combined with Timestamps and HMAC (Hash-based Message Authentication Code).**
- **Implementation Steps:**
 1. **Add Nonce & Timestamp:** Every outgoing request must include a **Cryptographic Nonce** (a single-use, cryptographically secure random string) and a precise UTC **Timestamp**.
 2. **Sign the Message:** Hash the payload, nonce, and timestamp using a shared secret key (HMAC) or asymmetric private key.
 3. **Server-Side Validation:**
 - **Time Window Check:** The server rejects any request whose timestamp falls outside an acceptable window (e.g., seconds).
 - **Nonce Verification:** The server tracks used nonces within that time window. If a nonce has already been processed, the request is instantly rejected as a duplicate.
- **Justification:** A replayed packet will carry an already-registered nonce or an expired timestamp. Even if an attacker intercepts and retransmits

the exact packet, the server's state validation will flag and discard it automatically.

Question 4: Secure Communication over Public Wi-Fi

4A. Risks of Unencrypted Public Wi-Fi

Accessing sensitive data over an open or unencrypted public Wi-Fi network exposes users to multiple attack vectors:

- **Eavesdropping / Packet Sniffing:** Attackers on the same unencrypted local network can capture raw network packets using software like Wireshark, exposing credentials, cookies, and sensitive text.
- **Man-in-the-Middle (MitM) Attacks:** Attackers can deploy rogue wireless access points ("Evil Twin" attacks) to position themselves between the user and the internet, intercepting or altering all communications.
- **Session Hijacking:** Unencrypted session cookies transmitted across the network can be intercepted, allowing attackers to hijack active user sessions without needing login passwords.

4B. Recommended Communication Method & Justification

- **Recommended Method: Transport Layer Security (TLS 1.3 / HTTPS) coupled with an Encrypted Virtual Private Network (VPN).**
- **Protection Mechanism:**
 1. **VPN Encapsulation:** A VPN establishes an encrypted IPsec or WireGuard tunnel from the user's device to a secure remote VPN server, encrypting all network layer traffic before it touches the local Wi-Fi router.
 2. **End-to-End Encryption (TLS 1.3):** Web requests are encrypted at the application layer, ensuring data remains unreadable even if the local network is compromised.
- **Justification:** Encrypted traffic renders packet sniffing useless; eavesdroppers on the public Wi-Fi see only unreadable ciphertext. Furthermore, TLS certificate verification ensures the user is connecting directly to legitimate servers rather than an attacker's MitM proxy.

Question 5: Data Integrity in Cloud Storage

5A. How Hashing Verifies Data Integrity

Cryptographic hash functions verify data integrity through deterministic, one-way mapping:

1. **Pre-Upload Hashing:** Prior to uploading a file to the cloud, the owner computes its cryptographic hash (e.g., using SHA-256) and stores this hash value in a secure local database.
2. **Integrity Auditing:** To verify the file later, the file is re-downloaded (or processed in-place via cloud API) and its hash is recalculated.
3. **Verification:** The new hash is compared against the pre-upload baseline hash. If the hashes match exactly, the data is intact. If even a single bit of data is altered or corrupted, the recalculated hash changes completely, confirming tampering or corruption.

5B. Additional Mechanism to Enhance Trust & Justification

- **Recommended Mechanism: HMAC (Hash-based Message Authentication Code) or Merkle Tree Audit Logging with Immutable Write-Once-Read-Many (WORM) Storage.**
- **Mechanism Details:**
 - Combine the cryptographic hash with a secret key held only by the data owner ().
 - Alternatively, construct a **Merkle Tree** (Hash Tree) of all stored data blocks, anchoring the Merkle Root hash in an immutable ledger or local secure system.
- **Justification:** Standard hashes stored alongside cloud data are vulnerable to malicious cloud insiders or compromised accounts—an attacker with file write permissions can modify the file and overwrite the plain hash. **HMAC** guarantees that an attacker cannot generate a valid replacement hash without the secret key, ensuring both **data integrity** and **source authenticity**. Merkle Trees enable efficient, scalable auditing of massive cloud datasets without downloading entire files.